

Experiment No: 8

Title: Deployment and Configuration options in Amazon (AWS).

Aim: To study Deployment and Configuration options in Amazon (AWS).

Theory:

Designing a deployment solution for application is a critical part of building a well-architected application on AWS. Based on the nature of your application and the underlying services (compute, storage, database, etc.) that it requires, we can use AWS services to create a flexible deployment solution that can be tailored to fit the needs of both application and our organization.

The constantly growing catalog of AWS services not only complicates the process of deciding which services will compose your application architecture, but also the process of deciding how you will create, manage, and update your application. When designing a deployment solution on AWS, you should consider how your solution will address the following capabilities:

- **Provision:** Create the raw infrastructure (Amazon EC2, Amazon Virtual Private Cloud [Amazon VPC], subnets, etc.) or managed service infrastructure (Amazon Simple Storage Service (Amazon S3), Amazon Relational Database Service [Amazon RDS], Amazon CloudFront, etc.) required for your application.
- **Configure:** Customize your infrastructure based on environment, runtime, security, availability, performance, network or other application requirements.
- **Deploy:** Install or update your application component(s) onto infrastructure resources, and manage the transition from a previous application version to a new application version.
- **Scale:** Proactively or reactively adjust the amount of resources available to your application based on a set of user-defined criteria.
- **Monitor:** Provide visibility into the resources that are launched as part of your application architecture. Track resources usage, deployment success/failure, application health, application logs, configuration drift, and more.

The task of designing a scalable, efficient, and cost-effective deployment solution should not be limited to the issue of how you will update your application version, but should also consider how you will manage supporting infrastructure throughout the complete application lifecycle. Resource provisioning, configuration management, application deployment, software updates, monitoring, access control, and other concerns are all important factors to consider when designing a deployment solution.

AWS provides a number of services that provide management capabilities for one or more aspects of your application lifecycle. Depending on your desired balance of control (i.e., manual management of resources) versus convenience (i.e., AWS management of resource

and the type of application, these services can be used on their own or combined to create a feature-rich deployment solution. This section will provide an overview of the AWS services that can be used to enable organizations to more rapidly and reliably build and deliver applications.

AWS provide following deployment services:

- AWS CloudFormation
- AWS Elastic Beanstalk
- AWS CodeDeploy
- Amazon Elastic Container Service
- Amazon Elastic Kubernetes Service
- AWS Ops-Works

AWS Elastic Beanstalk:

AWS Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services developed with Java, .NET, PHP, Node.js, Python, Ruby, Go, or Docker on familiar servers such as Apache, Nginx, Passenger, and IIS. Elastic Beanstalk is a complete application management solution, and manages all infrastructure and platform tasks on your behalf.

With Elastic Beanstalk, you can quickly deploy, manage, and scale applications without the operational burden of managing infrastructure. Elastic Beanstalk reduces management complexity for web applications, making it a good choice for organizations that are new to AWS or wish to deploy a web application as quickly as possible.

When using Elastic Beanstalk as your deployment solution, simply upload your source code and Elastic Beanstalk will provision and operate all necessary infrastructure, including servers, databases, load balancers, networks, and auto scaling groups. Although these resources are created on your behalf, you retain full control of these resources, allowing developers to customize as needed.

Table 2: AWS Elastic Beanstalk Deployment Features:

Capability	Description
Provision	Elastic Beanstalk will create all infrastructure components necessary to operate a web application or service that runs on one of its supported platforms. If you need additional infrastructure, this will have to be created outside of Elastic Beanstalk. Refer to Elastic Beanstalk Platforms for more details on the web application platforms supported by Elastic Beanstalk.
Configure	Elastic Beanstalk provides a wide range of options for customizing the resources in your environment. Refer to Configuring Elastic Beanstalk environments for more information about customizing the resources that are created by Elastic Beanstalk.
Deploy	Elastic Beanstalk automatically handles application deployments, and creates an environment that runs a new version of your application without impacting existing users. Refer to Deploying Applications to AWS Elastic Beanstalk for more details on application deployments with Elastic Beanstalk.
Scale	Elastic Beanstalk will automatically handle scaling of your infrastructure with managed auto scaling groups for your application instances. Refer to Auto Scaling Group for your Elastic Beanstalk Environment for more details about auto scaling with Elastic Beanstalk.
Monitor	Elastic Beanstalk offers built-in environment monitoring for applications including deployment success/failures, environment health, resource performance, and application logs. Refer to Monitoring an Environment for more details on full-stack monitoring with Elastic Beanstalk.

Elastic Beanstalk makes it easy for web applications to be quickly deployed and managed in AWS. The following example shows a general use case for Elastic Beanstalk as it is used to deploy a simple web application.

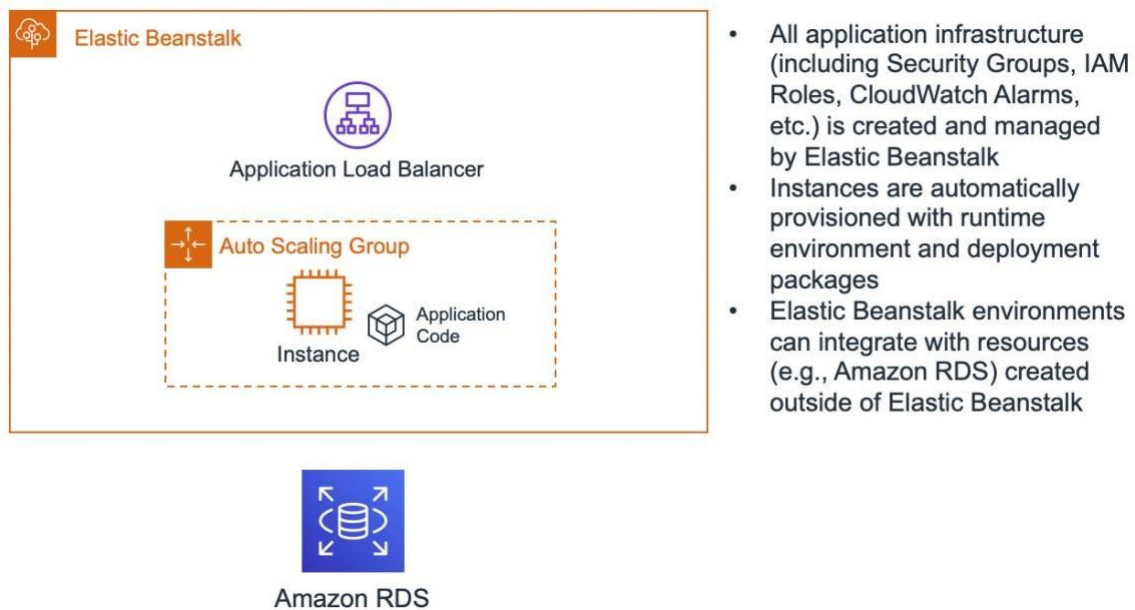


Figure: AWS Elastic Beanstalk use case

Configuration options:

Elastic Beanstalk defines a large number of configuration options that you can use to configure your environment's behavior and the resources that it contains. Configuration options are organized into namespaces like `aws:autoscaling:asg`, which defines options for an environment's Auto Scaling group.

The Elastic Beanstalk console and EB CLI set configuration options when you create an environment, including options that you set explicitly, and recommended values defined by the client. You can also set configuration options in saved configurations and configuration files. If the same option is set in multiple locations, the value used is determined by the order of precedence.

Configuration option settings can be composed in text format and saved prior to environment creation, applied during environment creation using any supported client, and added, modified or removed after environment creation. For a detailed breakdown of all of the available methods for working with configuration options at each of these three stages, read the following topics:

- Setting configuration options before environment creation
- Setting configuration options during environment creation
- Setting configuration options after environment creation

- **Settings applied directly to the environment** – Settings specified during a create environment or update environment operation on the Elastic Beanstalk API by any client, including the Elastic Beanstalk console, EB CLI, AWS CLI, and SDKs. The Elastic Beanstalk console and EB CLI also apply recommended values for some options that apply at this level unless overridden.
- **Saved Configurations** – Settings for any options that are not applied directly to the environment are loaded from a saved configuration, if specified.
- **Configuration Files (.ebextensions)** – Settings for any options that are not applied directly to the environment, and also not specified in a saved configuration, are loaded from configuration files in the .ebextensions folder at the root of the application source bundle.

Configuration files are executed in alphabetical order. For example, .ebextensions/01run.config is executed before .ebextensions/02do.config.

- **Default Values** – If a configuration option has a default value, it only applies when the option is not set at any of the above levels.

If the same configuration option is defined in more than one location, the setting with the highest precedence is applied. When a setting is applied from a saved configuration or settings applied directly to the environment, the setting is stored as part of the environment's configuration. These settings can be removed with the AWS CLI or with the EB CLI.

Settings in configuration files are not applied directly to the environment and cannot be removed without modifying the configuration files and deploying a new application version. If a setting applied with one of the other methods is removed, the same setting will be loaded from configuration files in the source bundle.

For example, say you set the minimum number of instances in your environment to 5 during environment creation, using either the Elastic Beanstalk console, a command line option, or a saved configuration. The source bundle for your application also includes a configuration file that sets the minimum number of instances to 2.

When you create the environment, Elastic Beanstalk sets the MinSize option in the aws:autoscaling:asg namespace to 5. If you then remove the option from the environment configuration, the value in the configuration file is loaded, and the minimum number of instances is set to 2. If you then remove the configuration file from the source bundle and redeploy, Elastic Beanstalk uses the default setting of 1.

Recommended values

The Elastic Beanstalk Command Line Interface (EB CLI) and Elastic Beanstalk console provide recommended values for some configuration options. These values can be different from the default values and are set at the API level when your environment is created.

Recommended values allow Elastic Beanstalk to improve the default environment configuration without making backwards incompatible changes to the API.

For example, both the EB CLI and Elastic Beanstalk console set the configuration option for EC2 instance type (InstanceType in the aws:autoscaling:launchconfiguration namespace). Each client provides a different way of overriding the default setting. In the console you can choose a different instance type from a drop down menu on the **Configuration Details** page of the **Create New Environment** wizard. With the EB CLI, you can use the --instance_type parameter for eb create.

Because the recommended values are set at the API level, they will override values for the same options that you set in configuration files or saved configurations. The following options are set:

Elastic Beanstalk console

- Namespace: aws:autoscaling:launchconfiguration
Option Names: IamInstanceProfile, EC2KeyName, InstanceType
- Namespace: aws:autoscaling:updatepolicy:rollingupdate
Option Names: RollingUpdateType and RollingUpdateEnabled
- Namespace: aws:elasticbeanstalk:application
Option Name: Application Healthcheck URL
- Namespace: aws:elasticbeanstalk:command
Option Name: DeploymentPolicy, BatchSize and BatchSizeType
- Namespace: aws:elasticbeanstalk:environment
Option Name: ServiceRole
- Namespace: aws:elasticbeanstalk:healthreporting:system
Option Name: SystemType and HealthCheckSuccessThreshold
- Namespace: aws:elasticbeanstalk:sns:topics
Option Name: Notification Endpoint
- Namespace: aws:elasticbeanstalk:sqs
Option Name: HttpConnections
- Namespace: aws:elb:loadbalancer
Option Name: CrossZone
- Namespace: aws:elb:policies
Option Names: ConnectionDrainingTimeout and ConnectionDrainingEnabled

EB CLI

- Namespace:
aws:autoscaling:launchconfiguration Option
Names: IamInstanceProfile, InstanceType
- Namespace: aws:autoscaling:updatepolicy:rollingupdate
Option Names: RollingUpdateType and RollingUpdateEnabled
- Namespace: aws:elasticbeanstalk:command
Option Name: BatchSize and BatchSizeType
- Namespace: aws:elasticbeanstalk:environment
Option Name: ServiceRole
- Namespace: aws:elasticbeanstalk:healthreporting:system
Option Name: SystemType
- Namespace: aws:elb:loadbalancer
Option Name: CrossZone
- Namespace: aws:elb:policies
Option Names: ConnectionDrainingEnabled

Conclusion: Thus, we have studied Deployment and Configuration options in Amazon(AWS).